

RTL DESIGN & VERIFICATION

CNN Accelerator 3-Channel 64x64 with 8 Filters

Hardware convolution accelerator with APB host access, line-buffered same-padding, and 8 parallel signed MAC datapaths.

PROJECT	cnn_3ch_64x64	INPUT	3 channels x 64x64, signed 8-bit
FILTERS	8 filters x 3x3x3	OUTPUT	8 x 64x64, signed 32-bit
INTERFACE	APB zero wait-state	VERIFICATION	10 tests, 327,680 checks

Convolution

Line Buffer

APB Slave

Signed MAC

Same Padding

Vivado XSim

1. Introduction

This project implements a hardware convolution accelerator in Verilog RTL, designed to compute a single convolutional layer of a CNN (Convolutional Neural Network). The accelerator accepts a 3-channel 64×64 input feature map (such as an RGB image or a quantized intermediate layer), applies 8 independent 3×3×3 convolution filters, and produces an 8×64×64 output feature map.

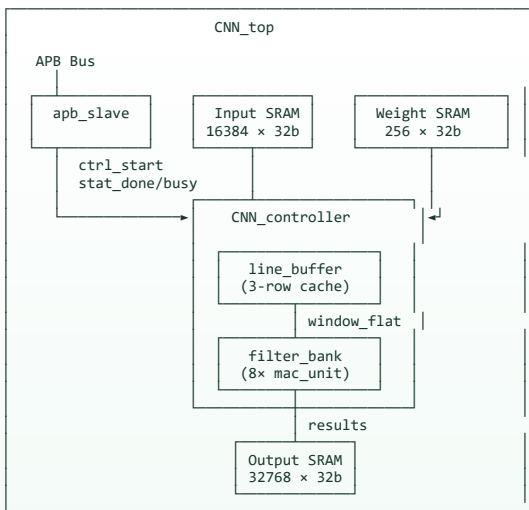
The design is interfaced via an APB (Advanced Peripheral Bus) slave, making it compatible with standard ARM Cortex-M based SoC integration. A host processor loads input pixels and filter weights into internal SRAMs via APB writes, triggers the computation, polls a status register for completion, and reads the output feature map via APB reads.

Key specifications:

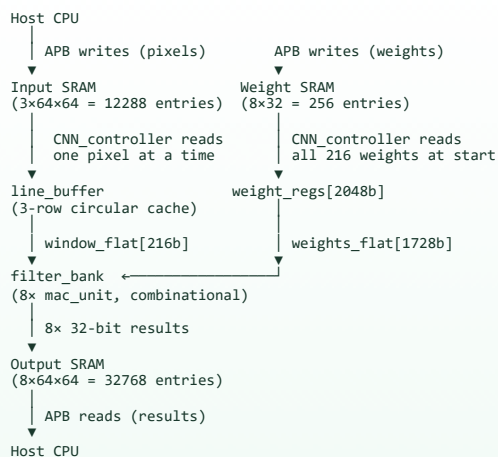
Parameter	Value
Input	3 channels × 64×64, signed 8-bit
Filters	8 filters × 3×3×3 kernel, signed 8-bit
Output	8 × 64×64, signed 32-bit
Padding	Same (zero-pad, output = input size)
Bias	None
Activation	None
Bus interface	APB (zero wait-state)
Target tool	Xilinx Vivado XSim

2. Architecture Overview

2.1 Top-Level Block Diagram



2.2 Data Flow



3. Module Descriptions

3.1 `sram.v` — Generic Synchronous SRAM

A parameterised synchronous memory macro. Write occurs on the rising clock edge when `we=1`. Read has **1-cycle latency**: address is presented in cycle N, data is valid in cycle N+1.

Parameters:

Parameter	Description
DEPTH	Number of words
AW	Address width (bits)
DW	Data width (bits)

Instantiated three times:

Instance	DEPTH	AW	DW	Contents
u_in_sram	16384	14	32	Input pixels (lower 8 bits used)
u_wt_sram	256	8	32	Weights (lower 8 bits used)
u_out_sram	32768	15	32	32-bit signed convolution results

3.2 mac_unit.v — 27-Input Signed MAC

Purely combinational. Computes one filter's dot product over the current $3 \times 3 \times 3$ input window:

```
result =  $\sum_{k=0..26}$  $signed(input[k]) × $signed(weight[k])
```

Each product is sign-extended to 32 bits before accumulation. The final 32-bit signed sum is the convolution output for one spatial position and one filter.

Ports:

Port	Width	Direction	Description
window_flat	$27 \times 8 = 216$	Input	Flattened $3 \times 3 \times 3$ input window
weight_flat	$27 \times 8 = 216$	Input	27 signed weights for one filter
result	32	Output	Signed 32-bit dot product

3.3 filter_bank.v — 8 Parallel MAC Units

Instantiates 8 `mac_unit`s. All share the **same** `window_flat` input (the current spatial position's $3 \times 3 \times 3$ window) but each receives a different set of 27 weights (one filter per unit). All 8 dot products are computed simultaneously in one clock cycle.

```
results_flat[0*32+:32] = window · weights[filter 0]  
results_flat[1*32+:32] = window · weights[filter 1]  
...  
results_flat[7*32+:32] = window · weights[filter 7]
```

3.4 line_buffer.v — 3-Row Circular Row Cache

Stores 3 consecutive input rows in three independent flat arrays (`lb_s0` , `lb_s1` , `lb_s2`). Each array holds one full row: `COLS×IN_CH` entries, indexed as `col×3 + ch` .

Read side forms the 3×3×3 sliding window combinatorially using intermediate mux wires. The `slot_rX` inputs select which physical array maps to kernel rows `kh=0`, `kh=1`, `kh=2`:

Signal	Kernel row	Meaning
<code>slot_r0</code>	<code>kh=0</code>	Row above current output row
<code>slot_r1</code>	<code>kh=1</code>	Current output row
<code>slot_r2</code>	<code>kh=2</code>	Row below current output row

Slot 3 zero-pad sentinel: When `slot_rX == 2'd3` , all reads for that kernel row return `{IDW{1'b0}}` (zero). This implements the top and bottom border padding without any conditional logic in the FSM.

Spatial zero-padding: Left (`kw=0`, `col==0`) and right (`kw=2`, `col==63`) border positions in `window_flat` are forced to zero in the `assign` statements.

window_flat layout: `window_flat[k*8 +: 8]` where `k = ch*9 + kh*3 + kw`

3.5 CNN_controller.v — FSM + Datapath Orchestrator

The central controller. Manages weight loading, row-by-row input streaming, convolution computation, and output writing via a 10-state FSM.

Registers:

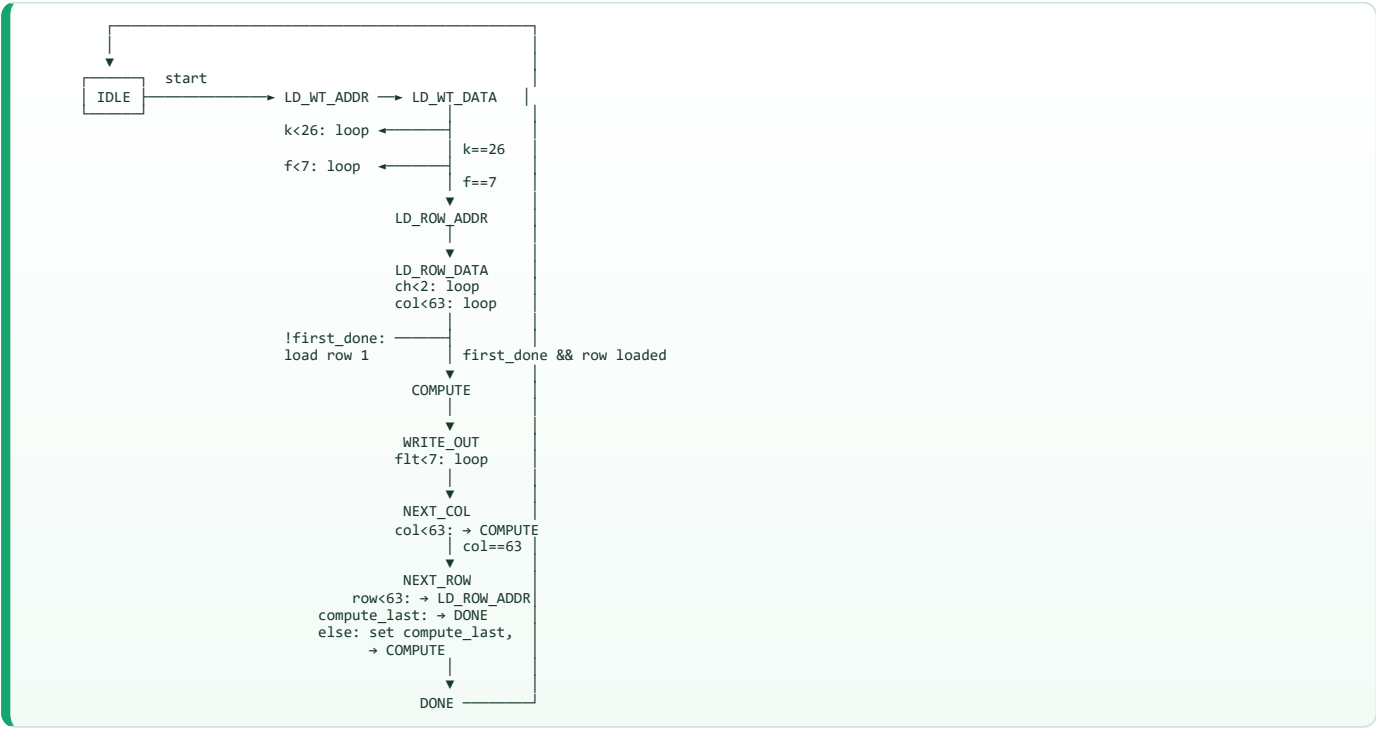
Register	Width	Purpose
<code>state</code>	4	FSM state
<code>row_cnt</code>	6	Input row currently being loaded (0..63)
<code>out_row_cnt</code>	6	Output row currently being computed (0..63)
<code>col_cnt</code>	6	Current column (shared for load and compute)
<code>ch_cnt</code>	2	Current input channel during row load (0..2)
<code>f_cnt</code>	3	Filter index during weight loading (0..7)
<code>k_cnt</code>	5	Kernel position during weight loading (0..26)
<code>flt_cnt</code>	3	Filter index during output write (0..7)
<code>slot_ptr</code>	2	Line buffer slot being written (0,1,2)
<code>slot_curr</code>	2	Line buffer slot holding <code>out_row_cnt</code> (= <code>out_row_cnt % 3</code>)
<code>first_done</code>	1	Set after row 0 pre-load; enables compute path
<code>compute_last</code>	1	Set for the final compute pass (output row 63)
<code>weight_regs</code>	2048	8 filters × 32 slots × 8 bits (flat register)

Slot assignment wires (combinational):

```
slot_r1 = slot_curr           // current output row
slot_r0 = (out_row_cnt == 0) ? 3 : slot_curr-1 // row above (or zero-pad)
slot_r2 = compute_last       ? 3 : slot_ptr    // row below (or zero-pad)
```

Weight storage: Weights are loaded from the weight SRAM into a 2048-bit flat register. Each filter occupies 32 slots (32×8=256 bits) even though only 27 weights are used — the stride of 32 allows `{f_cnt, k_cnt}` concatenation as the write index. `weights_flat` extracts 27 weights per filter.

3.6 FSM State Diagram



3.7 apb_slave.v — APB Bus Interface

Decodes incoming APB transactions and drives the appropriate SRAM or CSR signals. Zero wait-state (`pready` tied high). Fully combinational — no pipeline registers.

Address decode:

paddr[19:18]	Target	word_addr	pwrite	Operation
00	Input SRAM	paddr[15:2]	1	Write pixel
01	Weight SRAM	paddr[9:2]	1	Write weight
10	Output SRAM	paddr[16:2]	0	Read result
11, [3:0]=0	Control reg	—	1	Set start=pwdata[0]
11, [3:0]=4	Status reg	—	0	Read {busy, done}

Input SRAM address: `paddr[15:2]` gives a 14-bit word address (max 16383), covering the full 12288-element input space ($ch \cdot 4096 + row \cdot 64 + col$).

3.8 CNN_top.v — Top-Level + Arbiter

Connects all submodules and arbitrates SRAM ownership between the APB slave (idle/load/read phase) and the CNN controller (compute phase).

Arbitration logic:

```
// Input SRAM: controller drives addr during compute, APB write blocked
wire in_sram_we    = stat_busy ? 1'b0      : apb_in_we;
wire in_sram_addr  = stat_busy ? cc_in_addr : apb_in_addr;

// Weight SRAM: same pattern
wire wt_sram_we    = stat_busy ? 1'b0      : apb_wt_we;
wire wt_sram_addr  = stat_busy ? cc_wt_addr : apb_wt_addr;

// Output SRAM: always written by controller; addr muxed for APB read
wire out_sram_addr = stat_busy ? cc_out_addr : apb_out_addr;
```

When `stat_busy=1`, no APB write can corrupt the input or weight SRAMs.

4. Address Map

Input SRAM (`paddr[19:18] = 00`)

```
Word address = ch * 4096 + row * 64 + col
Byte address = (ch * 4096 + row * 64 + col) * 4
              = paddr[19:18]=00, paddr[15:2] = word_addr
```

Channel	Word address range
ch=0	0 – 4095
ch=1	4096 – 8191
ch=2	8192 – 12287

Weight SRAM (`paddr[19:18] = 01`)

```
Word address = f * 32 + k,    k = ch*9 + kh*3 + kw    (k < 27)
```

Filter	Word address range
f=0	0 – 31
f=1	32 – 63
...	...
f=7	224 – 255

Output SRAM (`paddr[19:18] = 10`)

```
Word address = f * 4096 + row * 64 + col
```

CSR (paddr[19:18] = 11)

Offset	Register	Bits
0xC0000	Control	[0] = start
0xC0004	Status	[0]=done, [1]=busy

5. Convolution Algorithm

Mathematical Definition

$$\text{output}[f][r][c] = \sum_{ch} \sum_{kh} \sum_{kw} \text{input}[ch][r+kh-1][c+kw-1] \times \text{weight}[f][ch][kh][kw]$$

where out-of-bounds accesses ($r+kh-1 < 0, \geq 64$, $c+kw-1 < 0, \geq 64$) are treated as zero (same-padding).

Same-Padding

With a 3×3 kernel and padding=1 on all sides:

$$\begin{aligned} \text{output_height} &= \text{input_height} - \text{kernel} + 2 \times \text{pad} + 1 = 64 - 3 + 2 + 1 = 64 \\ \text{output_width} &= \text{input_width} - \text{kernel} + 2 \times \text{pad} + 1 = 64 - 3 + 2 + 1 = 64 \end{aligned}$$

Output spatial dimensions match input spatial dimensions.

Software Reference (from tb_CNN.sv)

```
for (int f=0; f<8; f++)
  for (int r=0; r<64; r++)
    for (int c=0; c<64; c++) begin
      acc = 0;
      for (int ch=0; ch<3; ch++)
        for (int kh=0; kh<3; kh++)
          for (int kw=0; kw<3; kw++) begin
            ir = r + kh - 1;
            ic = c + kw - 1;
            if (ir>=0 && ir<64 && ic>=0 && ic<64)
              acc += $signed(input_map[ch][ir][ic])
                    * $signed(weight[f][ch][kh][kw]);
          end
      ref_out[f][r][c] = acc[31:0];
    end
end
```

6. Testbench

Strategy

The testbench (`tb_CNN.sv`) drives all interactions through the APB interface — identical to how a real host processor would use the accelerator. There are no backdoor memory writes or direct signal forcing.

Test Flow (per test case)

- 1. Generate random signed inputs (-128 to +127)
- 2. Generate random signed weights (-128 to +127)
- 3. Compute software golden reference (pure SystemVerilog loops)
- 4. Print 6x6 sample of input (all 3 channels)
- 5. Print complete filter 0 weights (3 channels x 3x3)
- 6. Print 6x6 sample of reference output (filter 0)
- 7. APB write: load 12,288 input pixels
- 8. APB write: load 216 weights (8 filters x 27)
- 9. APB write: set start=1
- 10. APB poll: read status until done=1
- 11. APB read: read all 32,768 output values
- 12. Compare each output against golden reference
- 13. Print per-test PASS/FAIL count

Coverage

Property	Value
Test cases	10
Input data	Random signed 8-bit per test
Weights	Random signed 8-bit per test
Elements checked	32,768 per test (all outputs)
Total checks	327,680
Timeout	5 seconds simulation time

7. Results

10

TEST CASES

327,680

ELEMENTS CHECKED

327,680

PASS

0

FAIL

All 10 random signed test cases passed with zero failures:

```
Test 1 result: 32768 PASS | 0 FAIL
Test 2 result: 32768 PASS | 0 FAIL
Test 3 result: 32768 PASS | 0 FAIL
Test 4 result: 32768 PASS | 0 FAIL
Test 5 result: 32768 PASS | 0 FAIL
Test 6 result: 32768 PASS | 0 FAIL
Test 7 result: 32768 PASS | 0 FAIL
Test 8 result: 32768 PASS | 0 FAIL
Test 9 result: 32768 PASS | 0 FAIL
Test 10 result: 32768 PASS | 0 FAIL

GRAND TOTAL (10 tests x 32768): 327680 PASS | 0 FAIL
```

8. Key Design Decisions

8.1 Row Pre-fetch Pipeline

Computing output row r requires input rows $r-1$, r , and $r+1$ simultaneously in the line buffer. A naive approach that loads row r and immediately computes output row r would have row $r+1$ missing. The FSM solves this by:

1. Pre-loading row 0, then immediately loading row 1 (before first compute)
2. After loading row r ($r \geq 1$), computing output row $r-1$
3. After the last load (row 63), computing output row 62, then computing output row 63 with `slot_r2 = zero-pad (slot 3)`

This introduces a one-row latency that is handled transparently by the FSM state machine.

8.2 Slot 3 Zero-Pad Sentinel

Rather than adding conditional logic in the FSM to detect top/bottom border rows, a virtual slot index `2'd3` is used. The line buffer returns all-zeros when any `slot_rX == 3`. This keeps the FSM clean and moves the boundary condition entirely into the line buffer's combinational mux.

8.3 Separate 2D Arrays in Line Buffer

Verilog-2001 3D arrays (`reg [W:0] arr [0:A][0:B][0:C]`) are not reliably simulated in XSim when accessed inside `assign` statements — the simulator does not always detect writes to inner dimensions as sensitivity events. The line buffer instead uses three flat 1D arrays (`1b_s0`, `1b_s1`, `1b_s2`) with explicit 4-way ternary mux wires for each read, ensuring all arrays appear on the RHS of combinational expressions and trigger re-evaluation.

8.4 Flat Weight Register

Weights are stored in a single 2048-bit register (`weight_regs`) using a stride of 32 slots per filter (even though only 27 weights are used). This allows the write address to be formed by simple bit concatenation: `{f_cnt, k_cnt}`, avoiding modulo arithmetic in the FSM.

8.5 APB Arbitration

The three SRAMs are shared between the APB slave (used during load and readback phases) and the CNN controller (used during compute). The arbiter in `CNN_top` uses the `stat_busy` flag as the select signal: when busy, the controller drives all SRAM addresses and APB writes are gated off. This prevents data corruption during compute with zero additional logic.

9. Limitations and Future Work

Limitation	Potential Enhancement
Fixed 64×64 spatial size	Parameterise ROWS/COLS and adjust SRAM depths
Fixed 3 input channels	Parameterise IN_CH, generalise line buffer depth
Fixed 8 filters	Parameterise NF
No bias addition	Add per-filter bias registers, accumulate after MAC
No activation function	Add ReLU/clip stage after output SRAM write
Same-padding only	Add valid-mode option (skip border rows/cols)
No output quantisation	Add right-shift + clamp to 8-bit for chained layers
Single inference at a time	Add double-buffering for ping-pong input loading
APB only (no DMA)	Add AXI stream or DMA interface for higher throughput
Weights reloaded every inference	Cache weights across multiple inferences of same layer